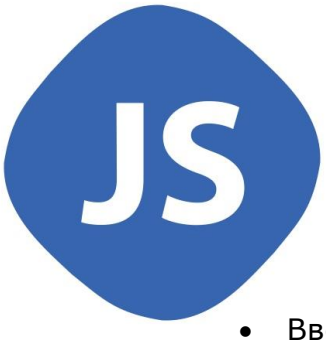


Объект URL

- Введение
- Создание объекта URL
- Свойство searchParams
- Итерация по свойству searchParams
- Объект URLSearchParams
- Кодирование в строках
- Кодирование в URLsearchParams
- Практика передачи данных с кодированием

Введение

HTTP — широко распространённый протокол передачи данных, изначально предназначенный для передачи гипертекстовых документов. Протокол HTTP предполагает использование клиент-серверной структуры передачи данных.

Клиентское приложение формирует запрос и отправляет его на сервер. В реальной жизни это означает, **пользователь открывает браузер и вводит запрос в адресную строку**, после этого браузер формирует тело запроса, автоматически добавляет нужные заголовки и отправляет на сервер.

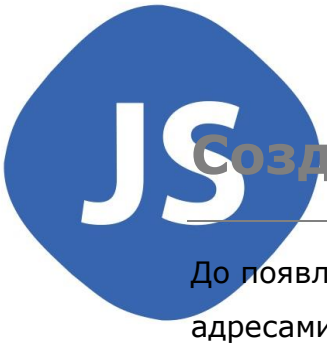
Сервер обрабатывает запрос, формируется ответ и отправляет обратно клиенту.

Запросы и ответы имеют довольно сложную структуру и будут подробнее рассмотрены в теме: **Асинхронный JavaScript**.

Ну, а в текущем уроке продолжим работу с адресной строкой и способами ее обработки. Полетели.

> ***Примечание***. Урок служит логическим продолжением урока **Объект Location**.





Создание объекта URL

До появления объектов URL, разработчики использовали для работы с URL-адресами **обычные строки**.

```
let url = 'http://js/target.php?course=javascript';
```

Встроенный класс URL предоставляет удобный интерфейс для **создания** и **разбора** URL-адресов.

Сетевые методы, требующие обычную строку URL, прекрасно будут работать и с объектом URL. Технически, разработчик не обязан использовать класс URL, но иногда он может быть очень удобным.

Синтаксис создания нового объекта URL:

```
new URL(url, [base]);
```

- **url** – **полный URL-адрес** или только путь, если указан второй параметр.
- **base** – необязательный **базовый URL**, если указан и аргумент **url** содержит только путь, то адрес будет создан относительно него.

Пример:

```
let url = new URL('http://js/target.php?course=javascript');
```

Объект URL даёт доступ к **компонентам URL**, поэтому это отличный способ разобрать URL-адрес.

example_1.

```
<script>

    // создаем объект URL

    let url = new
    URL('http://js/target.php?course=javascript&topic=bom&lesson=
    object-url#encoding');

    // раскладываем URL на компоненты
```

```
    document.write("href: <b>", url.href, "</b><br>");
    document.write("protocol: <b>", url.protocol, "</b><br>");
    document.write("host: <b>", url.host, "</b><br>");
    document.write("origin: <b>", url.origin, "</b><br>");
    document.write("pathname: <b>", url.pathname, "</b><br>");
    document.write("search: <b>", url.search, "</b><br>");
</script>
```

Исходя из синтаксиса объекта, следующие два URL **одинаковы**.

example_2.

```
<script>
    // создаем объект URL, вариант 1
    let url1 = new URL('http://js/target.php');
    // создаем объект URL, вариант 2
    let url2 = new URL('/target.php', 'http://js');
    // вывод объектов в консоль
    console.log(url1);
    console.log(url2);
</script>
```

Относительно существующего URL-адреса можно легко создать **новый URL**.

example_3.

```
<script>
    // создаем объект URL
    let url1 = new URL('http://js/javascript/');
    // создаем объекты URL относительно существующего
```

```
    let url2 = new URL('/bom', url1);
    let url3 = new URL('/object-url', url1);
    // вывод объектов в консоль
    console.log(url1);
    console.log(url2);
    console.log(url3);
</script>
```

Обращайте внимание на указанный при создании объекта **путь** (первый параметр). При всей внешней схожести, результат следующего примера принципиально иной. Проверяем, разбираемся.

example_4.

```
<script>
    // создаем объект URL
    let url1 = new URL('http://js/javascript/');
    // создаем объекты URL относительно существующего
    let url2 = new URL('bom', url1);
    let url3 = new URL('object-url', url2);
    // вывод объектов в консоль
    console.log(url1);
    console.log(url2);
    console.log(url3);
</script>
```



Свойство SearchParams

Допустим, перед нами стоит задача создать URL-адрес с заданными параметрами запроса, например,

```
let params = 'course=javascript&topic=bom&lesson=object-url#encoding';
```

Указать параметры можно в строке, формирующей URL-адрес:

```
let url = new URL('http://js/target.php?course=javascript&topic=bom&lesson=object-url#encoding');
```

И хотя такой способ формирования URL существует и часто используется, делать именно так не рекомендуется. Параметры должны быть правильно **закодированы** (подробно рассмотрим далее).

Для правильной работы с параметрами объект URL имеет свойство **searchParams** – объект типа **URLSearchParams**.

> ***Важно***. **Свойство** объекта URL – **searchParams** – в свою очередь является **объектом**. Объектом класса **URLSearchParams**.

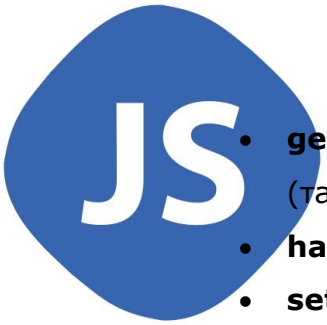
```
// создаем объект URL
let url = new URL('http://js');
console.log(url.searchParams); // объект URLSearchParams
```

Объект **URLSearchParams** (свойство **searchParams**) предоставляет удобные методы для работы с параметрами.

Методы объекта **URLSearchParams** (свойства **searchParams**):

- **append** (name, value) – добавить параметр по имени;
- **delete** (name) – удалить параметр по имени;
- **get** (name) – получить параметр по имени;





- **getAll** (name) – получить все параметры с одинаковым именем name (такое возможно, например: ?user=John&user=Евгений);
- **has** (name) – проверить наличие параметра по имени;
- **set** (name, value) – задать/заменить параметр;
- **sort** () – отсортировать параметры по имени, используется редко.

> ***Обратите внимание***. В следующих демонстрационных примерах обработка параметров строки запроса выполняется **серверным сценарием**, а значит проблем с кодировкой быть не должно. Про обработку URL-адреса **клиентским сценарием** дальше.

В следующих демонстрационных примерах разберем некоторые методы объекта **URLSearchParams** (свойства **searchParams**).

example_5. Метод get

```
<a id="link" href="#">
    Анализ строки запроса в серверном файле target.php
</a><p>
<script>
    // создаем объект URL, строку адреса с параметрами
    let url = new
    URL('http://js/target.php?course=javascript&topic=bom&lesson=
    object-url#encoding');
    console.log(url.searchParams); // объект URLSearchParams
    let course = url.searchParams.get("course");
    let topic = url.searchParams.get("topic");
    let lesson = url.searchParams.get("lesson");
    // вывод параметров в браузер
    document.write(`
        href: <b> ${url.href} </b><br>
```

```
        course: <b> ${course} </b><br>
        topic: <b> ${topic} </b><br>
        lesson: <b> ${lesson} </b><br>`
    );
    // обновляем атрибут href ссылки
    document.getElementById('link').setAttribute('href',
    url.href);
</script>
```

example_5. **target.php**

```
<h3>Сервер принял данные:</h3>
<?php
    echo "<pre>";
    print_r($_GET);
    echo "</pre>";
?>
```

example_6. Мктод append

```
<a id="link" href="#">
    Анализ строки запроса в серверном файле target.php
</a><p>
<script>
    // строка, представляющая адрес ссылки
    let href = 'http://js/target.php';
    // создаем объект URL
```

```
    let url = new URL(href);
    // добавляем параметры в объект URL
    url.searchParams.append("course", "javascript");
    url.searchParams.append("topic", "bom");
    url.searchParams.append("lesson", "object-url");
    // вывод ссылки в консоль
    console.log(url.href);
    // обновляем атрибут href ссылки
    document.getElementById('link').setAttribute('href',
    url.href);
</script>
```

example_6. **target.php**

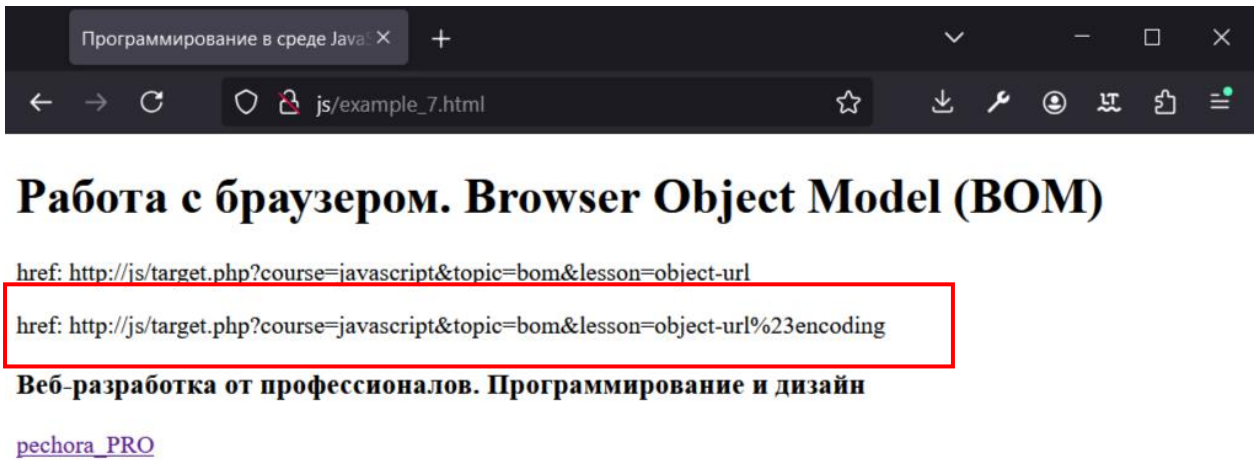
```
<h3>Сервер принял данные:</h3>
<?php
    echo "<pre>";
    print_r($_GET);
    echo "</pre>";
?>
```

Два следующих примера демонстрируют работу с идентификатором фрагмента. Обратите внимание на **результат** добавления идентификатора. Почему так происходит разберем в разделах, посвященных кодировкам.

example_7. Идентификатор фрагмента (hash)

```
<script>
    // создаем объект URL
    let url = new
```

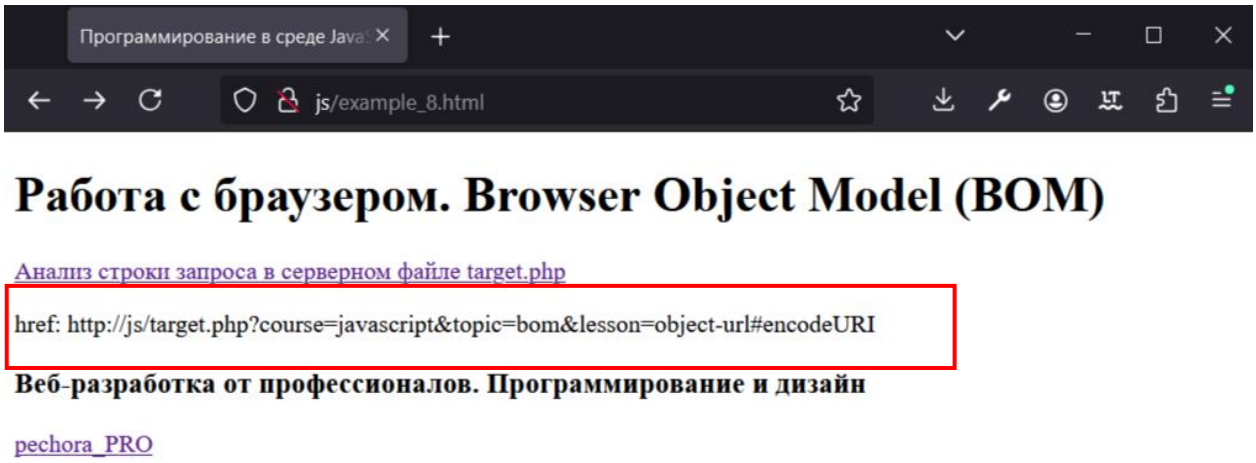
```
    URL('http://js/target.php?course=javascript&topic=bom&lesson=
    object-url');
    // вывод ссылки в браузер
    document.write("href: ", url.href, "<p>");
    // получаем значение параметра lesson
    let par = url.searchParams.get("lesson");
    // добавляем идентификатор фрагмента
    par = par + "#encoding";
    // обновляем значение параметра lesson
    url.searchParams.set("lesson", par);
    // вывод обновленной ссылки в браузер
    document.write("href: ", url.href);
</script>
```



example_8. Идентификатор фрагмента (hash)

```
<a id="link" href="#">
    Анализ строки запроса в серверном файле target.php
</a><p>
```

```
<script>
    // создаем объект URL
    let url = new
    URL('http://js/target.php?course=javascript&topic=bom&lesson=
    object-url#encoding');
    // добавляем идентификатор фрагмента
    url.hash = "#encodeURI";
    // вывод обновленной ссылки в браузер
    document.write("href: ", url.href);
    // обновляем атрибут href ссылки
    document.getElementById('link').setAttribute('href',
    url.href);
</script>
```



example_8. target.php

```
<h3>Сервер принял данные:</h3>
<?php
    echo "<h3>Серверная переменная REQUEST_URI</h3>";
    echo $_SERVER["REQUEST_URI"];
```

```
    echo "<h3>GET-параметры строки запроса</h3>";
    echo "<pre>";
    print_r($_GET);
    echo "</pre>";
    /*
        обратите внимание, hash-параметр:
        -- в адресной строке присутствует
        -- на сервер не отправляется
    */
?>
```

Итерация по свойству searchParams

Свойство **url.searchParams** представляет итерируемый объект **URLSearchParams**. Объект URLSearchParams, может непосредственно использоваться в структуре специального цикла **for...of.**

Получение имён всех параметров

Можно получить имена всех параметров, записанных в поиск, используя метод **keys**(). Результатом выбора будет **итератор**.

example_9.

```
<script>
    // создаем объект URL
    let url = new
    URL('http://js/target.php?course=javascript&topic=bom&lesson=
    object-url#encoding');
    // вывод в консоль строки параметров
```

```
    console.log(url.searchParams);
    // вывод в консоль итератора
    console.log(url.searchParams.keys());
    // вывод в консоль имен параметров
    for (let key of url.searchParams.keys()){
        console.log(key);
    }
</script>
```

Получение всех значений параметров

Метод **values**() получает значения параметров аналогично получению имён их параметров из примера выше.

example_10.

```
<script>
    // создаем объект URL
    let url = new
    URL('http://js/target.php?course=javascript&topic=bom&lesson=
    object-url#encoding');
    // вывод в консоль строки параметров
    console.log(url.searchParams);
    // вывод в консоль итератора
    console.log(url.searchParams.values());
    // вывод в консоль значений параметров
    for (let val of url.searchParams.values()){
        console.log(val);
    }
```

```
</script>
```

Получение всех параметров

По аналогии с предыдущими двумя методами **entries**() получает **итератор**, содержащий пары ключ/значение.

Следующие две строки **идентичны**:

```
for (var p of mySearchParams){...
for (var p of mySearchParams.entries()){...
```

example_11.

```
<script>
    // создаем объект URL
    let url = new
    URL('http://js/target.php?course=javascript&topic=bom&lesson=objec
    t-url#encoding');
    // вывод в консоль строки параметров
    console.log(url.searchParams);
    // вывод в консоль итератора
    console.log(url.searchParams.entries());
    // вывод в консоль: имя параметра -> значение
    // конструкции в циклах идентичны
    for (let [key, value] of url.searchParams.entries()){
        console.log(key, ' -> ', value);
    }
    console.log('-------');
    for (let [key, value] of url.searchParams){
        console.log(key, ' -> ', value);
    }
```

```
</script>
```

Объект URLSearchParams

URLSearchParams — это класс, предоставляющий удобный интерфейс для формирования **строки поисковых параметров**, которую потом можно использовать для формирования полного адреса. Все параметры в строке будут **закодированы** для безопасной вставки в адрес.

> ***Примечание***. Класс **URLSearchParams** — можно встретить как **часть класса URL**.

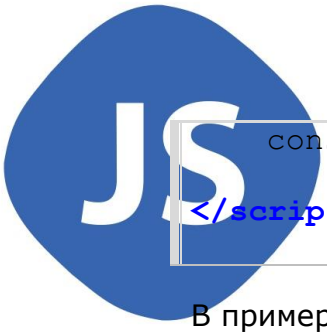
Когда нужно сформировать URL, включающий в себя строку поиска, то удобнее всего использовать **URLSearchParams**. Раньше для той же операции нужно было использовать цикл, в котором строка собиралась вручную, а также делать её безопасной для вставки в адрес с помощью **encodeURLComponent**().

Сейчас же можно использовать специальный класс и управлять им с помощью предоставляемых методов.

В демонстрационных примерах **example_12**-**example_14** выполним управление строкой адреса не через свойство **searchParams**, а через объект **URLSearchParams** (что, впрочем, одно и то же).

example_12.

```
<script>
    // создаем объект URL
    let url = new URL('http://js/target.php');
    console.log(url.searchParams); // объект URLSearchParams
    // создаем объект URLSearchParams
    let params = new URLSearchParams();
```



```
    console.log(params); // объект URLSearchParams
</script>
```

В примере **example_13** обратите внимание на фрагмент кода:

```
// выводим в консоль строковое представление объекта
console.log(params.toString());
```

example_13.

```
<script>
    // создаем объект URL
    let url = new URL('http://js/target.php');
    console.log(url.searchParams); // объект URLSearchParams
    // создаем объект URLSearchParams
    let params = new URLSearchParams();
    console.log(params); // объект URLSearchParams
    // добавляем к объекту params параметры
    params.append("course", "javascript");
    params.append("topic", "bom");
    params.append("lesson", "object-url");
    // выводим в консоль объект
    console.log(params); // объект URLSearchParams
    // выводим в консоль строковое представление объекта
    console.log(params.toString());
</script>
```

В примере **example_14** обратите внимание на фрагмент кода:

```
// приводим объект params к строке
// присваиваем свойству search строку параметров
```

```
url.search = params.toString();
```

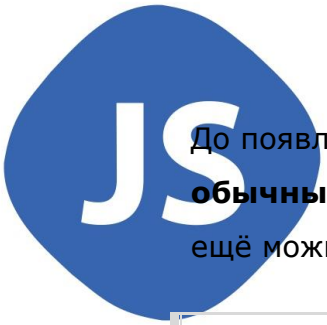
example_14.

```
<script>
    // создаем объект URL
    let url = new URL('http://js/target.php');
    console.log(url.searchParams); // объект URLSearchParams
    // создаем объект URLSearchParams
    let params = new URLSearchParams();
    console.log(params); // объект URLSearchParams
    // добавляем к объекту params параметры
    params.append("course", "javascript");
    params.append("topic", "bom");
    params.append("lesson", "object-url");
    // приводим объект params к строке
    // присваиваем свойству search строку параметров
    url.search = params.toString();
    console.log(params); // объект URLSearchParams
    console.log(url.searchParams); // объект URLSearchParams
</script>
```

Кодирование в строках

До сих пор мы отправляли данные в **серверный** сценарий. Сейчас попробуем выполнить обработку данных **клиентским** JS-сценарием. Разрешим вопросы кодирование строковых праметров адресной строки.

encodeURI, decodeURI



До появления объектов URL, разработчики использовали для URL-адресов **обычные строки**. Оперировать объектами URL гораздо удобнее, но строки всё ещё можно использовать. В некоторых случаях код с ними короче.

> ***Важно***. При использовании **строки**, разработчик вынужден сам позаботиться о кодировании пересылаемых данных.

Для работы с URL-строками есть встроенные функции:

- **encodeURI** – кодирует URL-адрес целиком или отдельно взятый параметр.
- **decodeURI** – декодирует URL-адрес целиком или отдельно взятый параметр.

Декодируем URL-адрес целиком.

example_15.

```
<a href="/target.html?topic=Управляющие
конструкции&lesson=Специальные циклы&section=Цикл for-of">
    Переход к файлу target.html
</a>
```

example_15. **target.html**

```
<script>
    // получаем строку параметров
    search = location.search.substr(1);
    // декодируем сразу всю строку
    search = decodeURI(search);
    // получаем массив параметров
    search = search.split('&');
```

```
    // выводим значения параметров
    for (par of search) {
        document.write(par.split("=")[1], "<br>");
    }
</script>
```

Декодируем отдельно каждый параметр.

example_16.

```
<a href="/target.html?topic=Управляющие
конструкции&lesson=Специальные циклы&section=Цикл for-of">
    Переход к файлу target.html
</a>
```

example_16. **target.html**

```
<script>
    // получаем массив параметров
    search = location.search.substr(1).split('&');
    // декодируем каждый параметр отдельно
    for (par of search) {
        let val = par.split("=")[1];
        // используем функцию decodeURI
        document.write(decodeURI(val), "<br>");
    }
</script>
```

Все работает. Но желательно брать процесс передачи данных полностью под свой контроль. В следующем примере выполним как **кодирование**, так **декодирование** данных самостоятельно.

example_17.

```
<a id="link" href="#">Переход к файлу target.html</a>
<script>
    // самостоятельно кодируем параметры
    let topic = encodeURI('Управляющие конструкции');
    let lesson = encodeURI('Специальные циклы');
    let section = encodeURI('Цикл for-of');
    // формируем атрибут href ссылки
    document.
    getElementById("link").
    setAttribute(
        "href",
        `target.html?topic=${topic}&lesson=${lesson}&section=${se
        ction}`
    );
</script>
```

example_17. target.html

```
<script>
    // получаем массив параметров
    search = location.search.substr(1).split('&');
    // декодируем каждый параметр отдельно
    for (par of search) {
        let val = par.split("=")[1];
        // используем функцию decodeURI
        document.write(decodeURI(val), "<br>");
```



```
}
</script>
```

Но есть небольшой (или большой, это как посмотреть) нюанс.

encodeURIComponent, decodeURIComponent

И нюанс этот состоит в том, что **некоторые специальные символы**, разрешенные в строке адреса могут использоваться и в **значениях** параметров.

Например:

- символ **&** может использоваться не как **разделитель** параметров, а как **часть значения** параметра.
- символ **#** может использоваться не как **разделитель** идентификатора фрагмента (хеша), а как **часть значения** параметра.

> ***Примечание***. В URL-адресе разрешены следующие специальные символы: , ?, =, &, #

example_18.

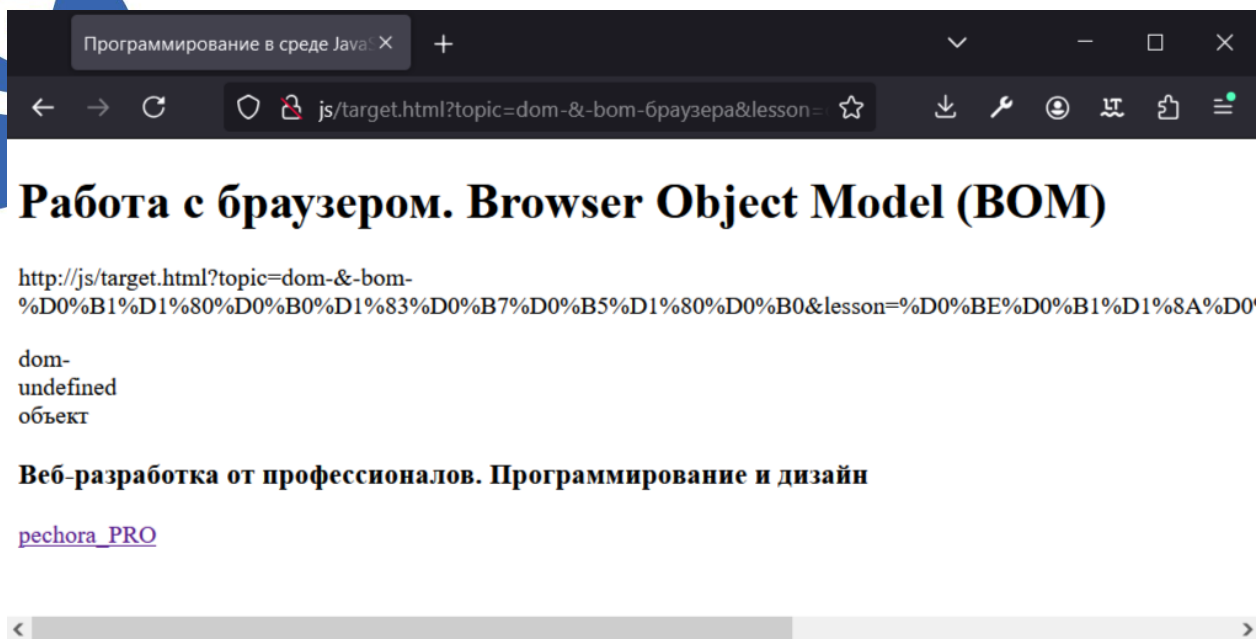
```
<a id="link" href="#">Переход к файлу target.html</a>
<script>
    // самостоятельно кодируем параметры
    // символ & используется не как разделитель параметров,
    // а как часть значения параметра
    let topic = encodeURI('dom-&-bom-браузера');
    let section = encodeURI('encodeURI&decodeURI');
    // символ # используется не как разделитель хеша,
```

```
    // а как часть значения параметра
    let lesson = encodeURI('объект#uri');
    // формируем атрибут href ссылки
    document.
    getElementById("link").
    setAttribute(
        "href",
        `target.html?topic=${topic}&lesson=${lesson}&section=${se
        ction}`
    );
</script>
```

example_18. target.html

```
<script>
    // строка адреса
    href = location.href;
    document.write(href, "<p>");
    // получаем массив параметров
    search = location.search.substr(1).split('&');
    // декодируем каждый параметр отдельно
    for (par of search) {
        let val = par.split("=")[1];
        document.write(decodeURI(val), "<br>");
    }
</script>
```

Получили не совсем то, что ожидали (да совсем не то...)



Таким образом, URL-параметры содержащие специальные символы должны быть закодированы специальным способом, чтобы не поломать форматирование.

Для этих целей используются специальные встроенные функции:

- **encodeURIComponent** – кодирует компонент URL, например, параметр, хеш, имя пути и т.п.
- **decodeURIComponent** – декодирует компонент URL.

Рассмотрим использование функций в демонстрационном примере **example_19**.

example_19.

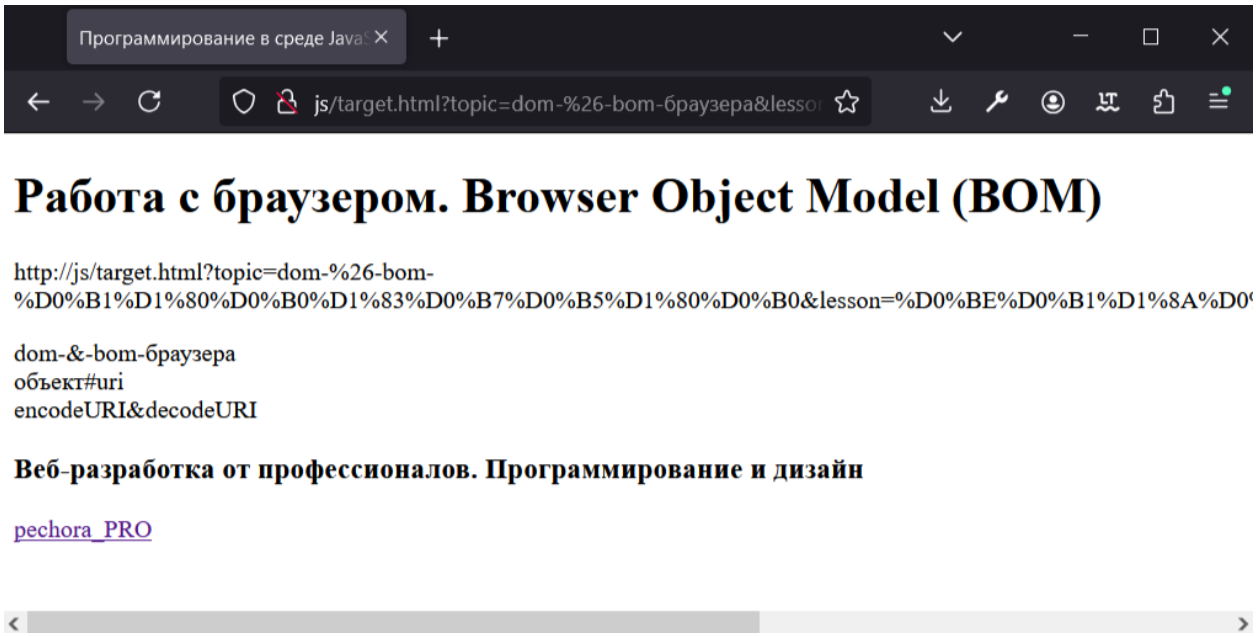
```
<a id="link" href="#">Переход к файлу target.html</a>
<script>
    // самостоятельно кодируем параметры
    // символ & используется не как разделитель параметров,
    // а как часть значения параметра
    let topic = encodeURIComponent('dom-&-bom-браузера');
    let section = encodeURIComponent('encodeURI&decodeURI');
```

```
    // символ # используется не как разделитель хеша,
    // а как часть значения параметра
    let lesson = encodeURIComponent('объект#uri');
    // формируем атрибут href ссылки
    document.
    getElementById("link").
    setAttribute(
        "href",
        `target.html?topic=${topic}&lesson=${lesson}&section=${se
        ction}`
    );
</script>
```

example_19. target.html

```
<script>
    // строка адреса
    href = location.href;
    document.write(href, "<p>");
    // получаем массив параметров
    search = location.search.substr(1).split('&');
    // декодируем каждый параметр отдельно
    for (par of search) {
        let val = par.split("=")[1];
        // обратите внимание
        // document.write(decodeURI(val), "<br>");
        document.write(decodeURIComponent(val), "<br>");
```

```
}
</script>
```



Разница между **encodeURIComponent**() и **encodeURI**() в следующем:

- **encodeURI** кодирует только символы, полностью запрещённые в URL;
- **encodeURIComponent** кодирует эти же символы плюс, в дополнение к ним, символы #, $, &, +, ,, /, :, ;, =, ? и @.

Примечание. Если для кодирования URL целиком функцию **encodeURI**() можно использовать, то функция **encodeURIComponent**() для этих же целей не годится.

Кодирование в URLsearchParams

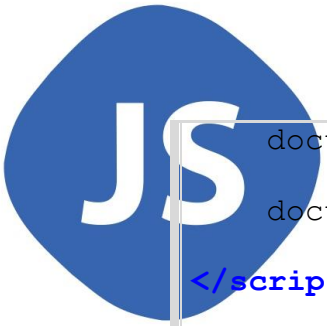
В целях обеспечения безопасности, все параметры, вставленные в строку URL-адреса через класс **URLSearchParams**() будут закодированы **автоматически**.

example_20.

```
<a id='link' href="#">Переход к файлу target.html</a>
<script>
    // создаем объект URL
    let url = new URL(location.origin + '/target.html');
    // добавляем параметры
    url.searchParams.append('topic', 'Управляющие конструкции');
    url.searchParams.append('lesson', 'Специальные циклы');
    url.searchParams.append('section', 'Цикл for-of');
    // устанавливаем атрибут href
    document.getElementById('link').setAttribute('href',
    url.href)
    // просмотр атрибута в консоли
    console.log(url.href);
</script>
```

example_20. target.html

```
<script>
    // создаем объект URL
    let url = new URL(location.href);
    // получаем параметры
    let topic = url.searchParams.get('topic')
    let lesson = url.searchParams.get('lesson')
    let section = url.searchParams.get('section')
    // выводим параметры в браузер
    document.write('topic = ', topic,"<br>");
```



```
    document.write('lesson = ', lesson,"<br>");
    document.write('section = ', section,"<br>");
</script>
```

Объединяем все вместе, добавляем **цикл** параметров, получаем код демонстрационного примера **example_21**.

example_21.

```
<a id="link" href="#">Переход к файлу target.html</a>
<script>
    // создаем объект URL
    let url = new URL (location.origin);
    // добавляем путь
    url.pathname = 'target.html';
    // добавляем параметры
    url.searchParams.append ('topic', 'dom-&-bom-браузера');
    url.searchParams.append ('lesson', 'объект-uri');
    url.searchParams.append ('section', 'encodeURI&decodeURI');
    // устанавливаем атрибут href
    document.getElementById('link').setAttribute('href',
    url.href);
</script>
```

example_21. **target.html**

```
<script>
    // создаем объект URL
    url = new URL(location.href);
    // разбираем объект URL
```

```
    for ([par, val] of url.searchParams.entries()) {
        document.write(`
        ${par} = ${val}<br>
    `);
    }
</script>
```

Практика передачи данных с кодированием

В завершении урока приведу практический пример сбора данных в параметр URL-адреса и безопасной передачи данных между страницами приложения.

example_22.

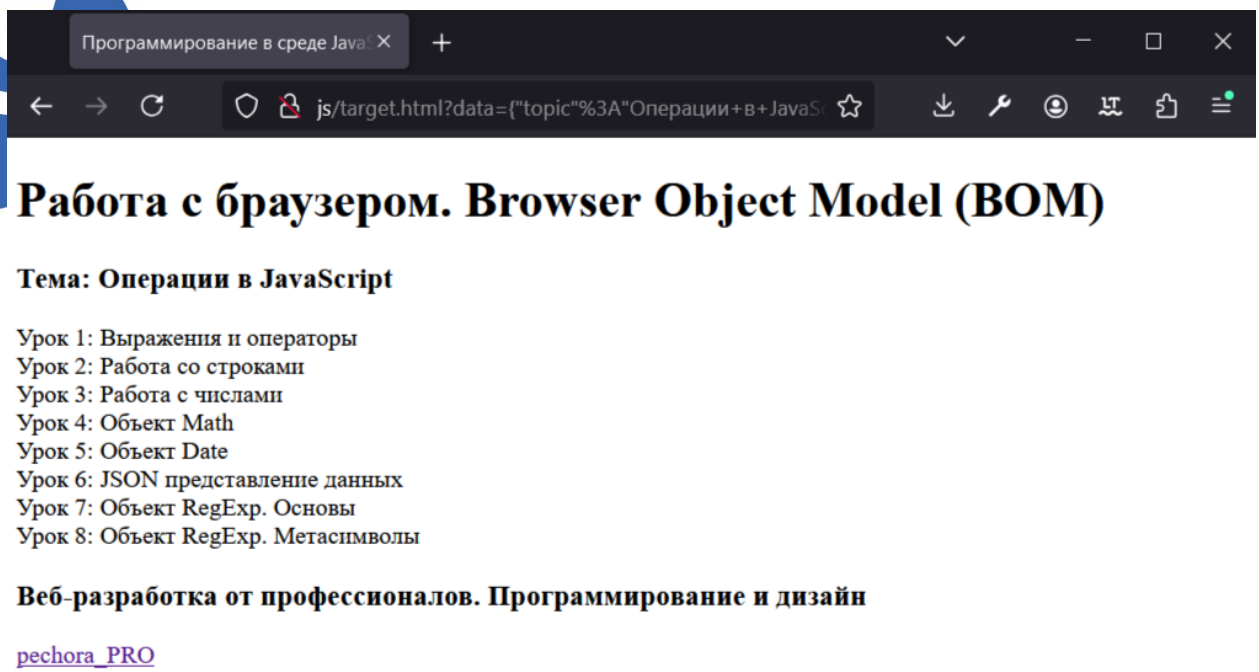
```
<a id="link" href="#">Переход к файлу target.html</a>
<ol>
    <!-- ... -->
    <!-- данные для передачи -->
    <li class="my-choice">
        <span>Операции в JavaScript</span>
        <ul>
            <li>Выражения и операторы</li>
            <li>Работа со строками</li>
            <li>Работа с числами</li>
            <li>Объект Math</li>
            <li>Объект Date</li>
            <li>JSON представление данных</li>
```

```
            <li>Объект RegExp. Основы</li>
            <li>Объект RegExp. Метасимволы</li>
        </ul>
    </li>
    <!-- ... -->
</ol>
<script>
    // выбор пунктов списка в коллекцию
    lis = document.querySelectorAll("ol > li");
    // перебор коллекции
    for (let li of lis) {
        if (li.className == "my-choice") {
            let topic = li.firstElementChild.innerText;
            let ul = li.lastElementChild;
            // формируем объект для передачи
            data = JSON.stringify({
                topic: topic,
                lesson: ul.innerText
            });
            // создаем объект URL
            url = new URL(location.origin + '/target.html');
            // добавляем параметр data
            url.searchParams.append('data', data);
        }
    }
    // формируем атрибут href ссылки
```

```
    document.getElementById("link").setAttribute("href",
    url.href);
</script>
```

example_22. target.html

```
<script>
    // создаем объект URL
    url = new URL(location.href);
    // получение параметра объекта URL
    data = JSON.parse(url.searchParams.get('data'));
    // извлекаем данные параметра
    topic = data.topic;
    arr_lesson = data.lesson.split("\n");
    // вывод полученных данных в браузер
    document.write("<h3>Тема: ", topic, "</h3>");
    arr_lesson.forEach( (lesson, key) => {
        document.write(`
            Урок ${++key}: ${lesson}<br>
        `)
    })
</script>
```



Больше примеров и практики в **самостоятельных работах**:

- Создание динамических ссылок. Безопасная передача данных в URL.
- Формирование URL-адреса в цикле.
- Формирование относительных адресов для объекта URL.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.



К **курсу**:

- **Дополнительный практикум.**

